

tmux documentation

CodeGraph — code-intelligence knowledge graph for vasic-digital tmux

Revision: 1 **Last modified:** 2026-05-21T00:00:00Z **Mandate:** constitution/
Constitution.md §11.4.78 + user mandate (2026-05-21) **Project anchor:** A18 in
[Fixed.md](#)

§1 — What is CodeGraph and why is it mandatory here?

[CodeGraph](#) is a 100%-local SQLite knowledge graph of your project's source code, exposed to AI coding agents over MCP (Model Context Protocol). No cloud, no external API, no telemetry. The MCP server lets agents (Claude Code, OpenCode, Kimi CLI, Crush, Qwen Code) ask structured questions about the codebase ("which files import this function?", "what's the call graph?", "list nodes by kind") without burning context on file reads.

It is mandatory for every project worked on by AI coding agents per constitution submodule §11.4.78. Every consumer (this project included) **MUST** install it, initialize it, wire it for every supported CLI agent, and verify the wiring with anti-bluff tests.

§2 — Operator-facing install (one-time per machine)

```
# 1. Prereqs: Node 20+ on PATH; npm prefix MUST be user-writable
#    (per §11.4.78 — NO sudo).
node --version      # ≥ 20
npm --version
npm config get prefix # must be writable by your user

# 2. Install the CLI globally.
npm install -g @colbymchenry/codegraph
codegraph --version # ≥ 0.6.8 expected

# 3. (Per project) initialise + index.
cd /path/to/repo
bash scripts/codegraph_reindex.sh # canonical regeneration
                                mechanism
```

If `npm config get prefix` points at a root-owned path (`/usr/local` unless reconfigured), fix it without `sudo` first:

```
mkdir -p ~/.npm-global
npm config set prefix ~/.npm-global
export PATH=~/.npm-global/bin:$PATH # add to ~/.bashrc /
                                   ~/.zshrc
```

§3 — What's tracked in this repo

Path	Tracked?	Purpose
<code>.codegraph/config.json</code>	✓	Index include/exclude — secret-safe per §11.4.10
<code>.codegraph/codegraph.db</code>	gitignored	Per-machine SQLite index. Regenerate via <code>scripts/codegraph_reindex.sh</code>
<code>.codegraph/codegraph.db-wal / -shm</code>	gitignored	SQLite WAL/SHM journals
<code>.codegraph/.gitignore</code>	✓	CodeGraph-managed; covers the above
<code>.gitignore-meta/codegraph-db.yaml</code>	✓	§11.4.77 regeneration manifest (so a fresh clone can rebuild without operator guesswork)
<code>scripts/codegraph_reindex.sh</code>	✓	Idempotent regeneration script

§4 — Secret-exclusion contract (§11.4.10)

The config's `exclude` list MUST hold (verified by test 20 T3):

```
/**/*.env  **/*.env.*  **/*.env  **/*.pem  **/*.key  **/*.crt
**/id_rsa*  **/id_ed25519*  **/secrets/**
constitution/**  Containers/**  tmux/**
```

The first nine entries prevent credential ingest per §11.4.10. The three submodule paths honour §11.4.28 owned-submodule decoupling — each owned submodule maintains its own CodeGraph index in its own cycle; the parent project never reaches into them. `tmux/` is the upstream third-party submodule and must stay unindexed (we never claim authorship of upstream code).

§5 — MCP wiring per CLI agent (§11.4.78)

Every supported CLI agent has its CodeGraph MCP server config wired:

Agent	Config path	Scope	Status
Claude Code (this project's primary)	<code>.mcp.json</code>	project	✓ committed
OpenCode	<code>~/.config/opencode/opencode.json</code>	host	✓ if installed
Kimi CLI	<code>~/.kimi/mcp.json</code>	host	✓ if installed
Crush	<code>.crush.json</code>	project	✓ committed
Qwen Code	<code>.qwen/settings.json</code>	project	✓ committed

Every config references the bare `codegraph` command on `PATH` — no hardcoded host paths — so configs are portable across machines (per §11.4.78). Test 22 parses each config + verifies the bare-`PATH` constraint mechanically.

§6 — Anti-bluff verification (§11.4.78 + §11.4)

Three test scripts gate the integration end-to-end:

- `scripts/tests/20_codegraph_installed.sh` — CLI $\geq 0.6.0$ on PATH, `config.json` present + valid JSON, all 12 required exclude patterns present, `.gitignore` covers the DB, §11.4.77 regen manifest + executable script.
- `scripts/tests/21_codegraph_index_present.sh` — DB file present
 - non-trivial size, `codegraph status` reports a non-zero node count (positive runtime evidence per §11.4.5), §11.4.77 regen stamp file records the node count.
- `scripts/tests/22_codegraph_mcp_wired.sh` — each of the 5 agent configs JSON-parsed; project-scoped configs MUST exist; host-scoped configs SKIP-with-reason if the agent isn't installed (§11.4.3 topology dispatch); every command field references the bare codegraph (no hardcoded paths); `codegraph serve --mcp` spawns cleanly + stays alive.

Plus three paired §103 layer-4 mutations:

- **M16** strips one required secret-exclusion from `config.json` → test 20 T3 must FAIL.
- **M17** strips the codegraph entry from `.mcp.json` → test 22 T1 must FAIL.
- **M19** strips the AUDIT-2 `kill`-shorthand alias from the wrapper (related cycle work) → test 23 T3 must FAIL.

§7 — Unforgeable-challenge note (§11.4.78)

The §11.4.78 mandate calls for an "unforgeable challenge" — a fact obtainable only by calling a CodeGraph MCP tool, so an agent answering from its own file-reading tools cannot produce a false PASS. The version of that test that drives a real CLI agent non-interactively is classified AUTONOMOUS_DEIGNED per §11.4.52 (documented but not yet runnable in CI). The mechanical seam exists (test 22 T7 spawns the MCP server and asserts it stays alive); the agent-driven layer lands in a follow-up cycle when a headless agent harness is wired.

§8 — Operator-path examples

```
# Force a full reindex (e.g., after large refactor).
codegraph index .

# Incremental sync (run after editing a few files).
codegraph sync .

# Show index stats.
codegraph status

# Query the knowledge graph (CLI surface — same as MCP
  `codegraph_query`).
codegraph query 'function:_apply_host_color'

# List indexed files.
codegraph files
```

```
# Build LLM-friendly context for a task.
codegraph context 'understand how per-session isolation works'

# Open the interactive graph in browser.
codegraph visualize
```

§9 — Honest gaps (§11.4.6)

- **Shell parser not shipped with CodeGraph 0.6.8.** This project is primarily Bash (60+ shell scripts), and CodeGraph indexes only the one C file (`scripts/oom_set.c`) — node count = 6. This is the HONEST current state, not bluff. Adding shell tree-sitter support would be an upstream contribution (out of scope here per §11.4.74).
- **No agent-driven end-to-end test yet.** See §7 above — AUTONOMOUS_DESIGNED per §11.4.52 carve-out.

§10 — Troubleshooting

Symptom	Diagnosis
<code>codegraph</code> : command not found	npm prefix not on PATH. Add <code>\$(npm config get prefix)/bin</code> to PATH (no sudo).
<code>npm install -g</code> fails with EACCES	npm prefix is root-owned. Fix per §2 above (set user-writable prefix; no sudo).
<code>codegraph status</code> reports 0 nodes after init	Re-run <code>bash scripts/codegraph_reindex.sh</code> . If still 0, file types may be filtered — check <code>include</code> in <code>config.json</code> .
MCP server "stays alive but agent sees no tools"	Verify the agent's config points at <code>codegraph serve --mcp</code> (test 22 catches this).
Containers/ or constitution/ paths leaking into the index	<code>exclude</code> list incomplete — test 20 T3 catches this; re-add the missing pattern + reindex.

§11 — Composition with other constitution anchors

- §11.4.10 (credentials) — secret-exclusion contract above.
- §11.4.28 (owned-submodule decoupling) — never index a sibling owned submodule from the parent index.
- §11.4.30 (.gitignore + no-versioned-build-artifacts) — `.db` ignored.
- §11.4.65 (universal Markdown export) — this doc has HTML + PDF siblings refreshed by `scripts/export_docs.sh`.
- §11.4.74 (catalogue-first) — CodeGraph reused from npm (not reimplemented); upstream extensions land in their own cycles.
- §11.4.77 (regeneration mechanism) — `codegraph-db.yaml` manifest.
- §11.4.78 (this anchor).